

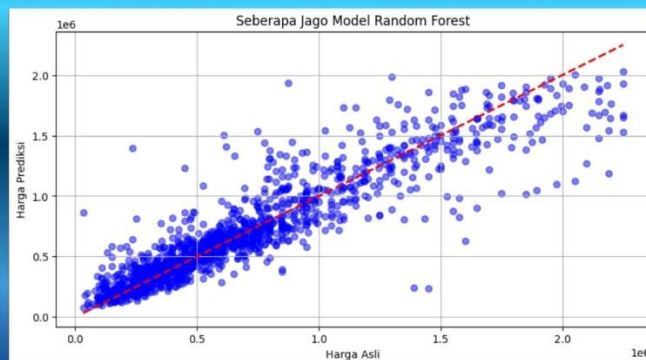
Summary

Proyek ini mengenai prediksi mobil yang ada di India. Saya menggunakan dataset yang ada di Kaggle yang judulnya 'Used Car Dataset'. Pada proyek ini, saya menggunakan model reandom forest regressor karena target dari dataset yang saya miliki itu numerik. Alhamdulillah, akurasi model yang saya buat mencapai kurang lebih 83%. Dengan menyelesaikan proyek ini, saya belajar banyak mengenai bagaimana cara membangun model agar bisa mencapai akurasi yang optimal.

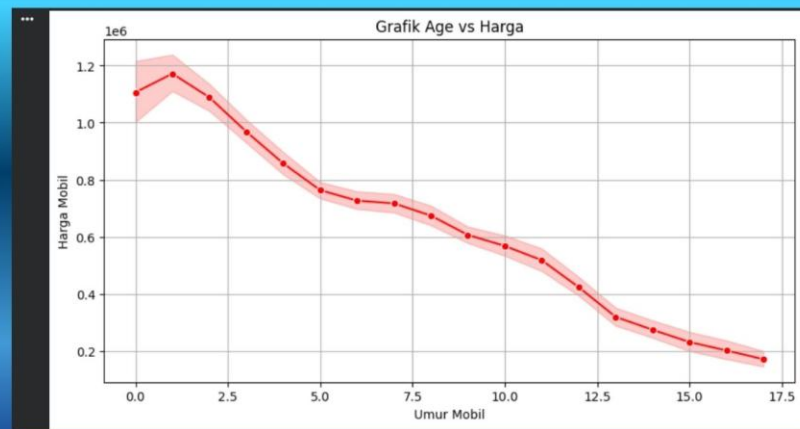
Feature Importance



Akurasi model



Korelasi Antara Harga Mobil dengan Umur Mobil



Deskripsi

- Menyiapkan library yang akan digunakan

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, r2_score
```

- Import pandas as pd digunakan untuk memanipulasi data. Misalnya untuk membuat dataframe, membaca dataset yang kita punya, dan lain sebagainya
- Import matplotlib.pyplot as plt digunakan untuk visualisasi data.
- Import seaborn as sns digunakan untuk visualisasi data juga, hanya saja kalau seaborn itu lebih modern dan settingan nya tidak sebanyak matplotlib.pyplot
- From sklearn.model_selection import train_test_split digunakan untuk membagi data latih dan data uji
- From sklearn.preprocessing import StandardScaler digunakan untuk membuat dataset kita memiliki rata-rata = 0 dan standar deviasi = 1
- From sklearn.ensemble import RandomForestRegressor digunakan untuk membuat model Random Forest Regressor. Kita menggunakan regressor karena target yang akan kita prediksi berbentuk numerik bukan kategorikal
- From sklearn.metrics import mean_absolute_error, r2_score digunakan untuk mengevaluasi model yang kita punya. Mean_absolute_error digunakan untuk melihat rata-rata selisih (jarak) antara data asli dengan data prediksi. R2_score digunakan untuk melihat seberapa bagus model kita memahami pola.

- Membaca dataset yang kita punya dan menampilkan 5 baris pertama dari dataset

```
df = pd.read_csv('used_car_dataset.csv')
df.head()
```

	Brand	model	Year	Age	kmDriven	Transmission	Owner	FuelType	PostedDate	AdditionInfo	AskPrice
0	Honda	City	2001	23	98,000 km	Manual	second	Petrol	Nov-24	Honda City v teck in mint condition, valid gen...	₹ 1,95,000
1	Toyota	Innova	2009	15	190000.0 km	Manual	second	Diesel	Jul-24	Toyota Innova 2.5 G (Diesel) 7 Seater, 2009, D...	₹ 3,75,000
2	Volkswagen	VentoTest	2010	14	77,246 km	Manual	first	Diesel	Nov-24	Volkswagen Vento 2010-2013 Diesel Breeze, 2010...	₹ 1,84,999
3	Maruti Suzuki	Swift	2017	7	83,500 km	Manual	second	Diesel	Nov-24	Maruti Suzuki Swift 2017 Diesel Good Condition	₹ 5,65,000
4	Maruti Suzuki	Baleno	2019	5	45,000 km	Automatic	first	Petrol	Nov-24	Maruti Suzuki Baleno Alpha CVT, 2019, Petrol	₹ 6,85,000

- Melihat berapa banyak baris dan kolom yang dimiliki dataset

```
df.shape  
  
(9582, 11)
```

Dataset kita memiliki 9.582 baris dan 11 kolom

- Menampilkan tipe data dari setiap kolom (feature)

```
df.info()  
  
... <class 'pandas.core.frame.DataFrame'>  
RangeIndex: 9582 entries, 0 to 9581  
Data columns (total 11 columns):  
#   Column          Non-Null Count  Dtype  
---  ---  
0   Brand           9582 non-null   object  
1   model           9582 non-null   object  
2   Year            9582 non-null   int64  
3   Age             9582 non-null   int64  
4   kmDriven        9535 non-null   object  
5   Transmission    9582 non-null   object  
6   Owner           9582 non-null   object  
7   FuelType        9582 non-null   object  
8   PostedDate      9582 non-null   object  
9   AdditionInfo    9582 non-null   object  
10  AskPrice        9582 non-null   object  
dtypes: int64(2), object(9)  
memory usage: 823.6+ KB
```

- Ada beberapa kolom yang memiliki value numerik namun tipe datanya terdeteksi sebagai object. Ini terjadi karena di dalam kolom tersebut isinya bukan angka saja, melainkan karakter-karakter lain selain angka. Oleh karena itu, kita akan menghapus karakter-karakter yang membuat kolom (feature) tersebut menjadi object

```
# Menghapus karakter yang bukan angka di dalam kolom kmDriven
df['kmDriven'] = df['kmDriven'].astype(str)
df['kmDriven'] = df['kmDriven'].str.replace(r'\D', '', regex=True)

# Mengubah tipe data kolom kmDriven menjadi numerik
df['kmDriven'] = pd.to_numeric(df['kmDriven'], errors='coerce')

# Menghapus karakter yang bukan angka di dalam kolom AskPrice
df['AskPrice'] = df['AskPrice'].astype(str)
df['AskPrice'] = df['AskPrice'].str.replace(r'\D', '', regex=True)

# Mengubah tipe data kolom AskPrice menjadi numerik
df['AskPrice'] = pd.to_numeric(df['AskPrice'], errors='coerce')
```

- Kita cek kembali tipe data dari setiap kolom (feature)

```
df.info()

... <class 'pandas.core.frame.DataFrame'>
RangeIndex: 9582 entries, 0 to 9581
Data columns (total 11 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   Brand            9582 non-null   object
1   model            9582 non-null   object
2   Year             9582 non-null   int64
3   Age              9582 non-null   int64
4   kmDriven         9535 non-null   float64
5   Transmission     9582 non-null   object
6   Owner            9582 non-null   object
7   FuelType         9582 non-null   object
8   PostedDate       9582 non-null   object
9   AdditionInfo     9582 non-null   object
10  AskPrice         9582 non-null   int64
dtypes: float64(1), int64(3), object(7)
memory usage: 823.6+ KB
```

Terdapat perubahan tipe data dari kolom kmDriven dan AskPrice

- Menghapus kolom yang menurut saya tidak berguna

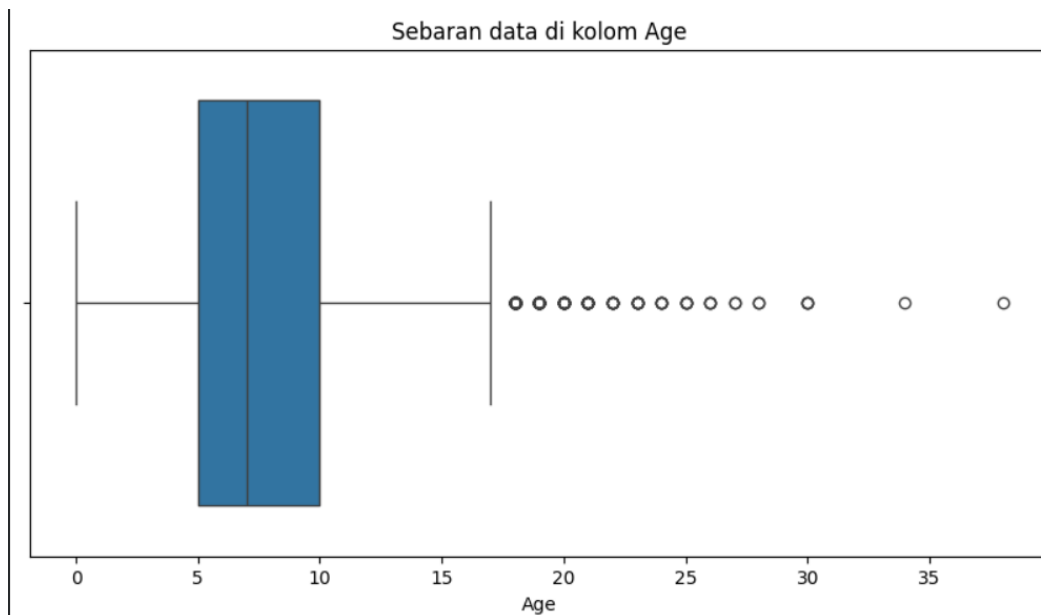
```
df = df.drop(columns=['PostedDate', 'AdditionInfo', 'Year'])
df.head()
```

	Brand	model	Age	kmDriven	Transmission	Owner	FuelType	AskPrice
0	Honda	City	23	98000.0	Manual	second	Petrol	195000
1	Toyota	Innova	15	1900000.0	Manual	second	Diesel	375000
2	Volkswagen	VentoTest	14	77246.0	Manual	first	Diesel	184999
3	Maruti Suzuki	Swift	7	83500.0	Manual	second	Diesel	565000
4	Maruti Suzuki	Baleno	5	45000.0	Automatic	first	Petrol	685000

- Mengecek outliers pada dataset kita. Kita harus mengecek outlier karena jika terdapat outlier didalam dataset kita, maka akan sangat mempengaruhi akurasi model kita. Model kita akan memiliki akurasi yang buruk, jika terdapat outlier.

```
# Mengecek outliers
cols = ['Age', 'kmDriven', 'AskPrice']

for i in cols:
    plt.figure(figsize=(10, 5))
    sns.boxplot(x=df[i])
    plt.title(f'Sebaran data di kolom {i}')
    plt.show()
```



Kita bisa lihat disini bahwa kolom Age memiliki outlier yang lumayan banyak. Pada gambar diatas, outlier disimbolkan dengan lingkaran-lingkaran kecil yang berjejer ke samping.

- Memfilter dataset yang tidak memiliki outlier dengan metode IQR (Interquartile Range).

```
Q1 = df[cols].quantile(0.25)
Q3 = df[cols].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - (1.5 * IQR)
upper_bound = Q3 + (1.5 * IQR)

condition = ~(df[cols] < lower_bound) | (df[cols] > upper_bound)
df_copy = df[condition].copy()
```

Setelah itu, kita copy dataset yang sudah di filter. Tujuan meng copy dataset ini agar dataset asli yang kita miliki itu tidak rusak/tidak berubah. Data yang akan kita rubah itu data copy an dari dataset asli kita

- Perbandingan jumlah baris dan kolom sebelum dibersihkan dan setelah dibersihkan

```
print(f'Jumlah data asli : {df.shape}')
print(f'Jumlah data bersih : {df_copy.shape}')
```

... Jumlah data asli : (9582, 8)
Jumlah data bersih : (7722, 8)

- Mengecek nilai kosong yang ada didalam dataset

```
df_copy.isnull().sum()
```

	0
Brand	0
model	0
Age	0
kmDriven	43
Transmission	0
Owner	0
FuelType	0
AskPrice	0

dtype: int64

- Menghapus nilai kosong, kemudian kita cek kembali apakah masih ada nilai kosong

```
df_copy = df_copy.dropna()
df_copy.isnull().sum()
```

...	0
Brand	0
model	0
Age	0
kmDriven	0
Transmission	0
Owner	0
FuelType	0
AskPrice	0

dtype: int64

- Mengecek nilai yang duplikat pada dataset yang kita punya

```
df_copy.duplicated().sum()

np.int64(623)
```

- Menghapus nilai yang duplikat

```
df_copy = df_copy.drop_duplicates()
df_copy.duplicated().sum()

np.int64(0)
```

- Memisahkan fitur kategorikal untuk dilakukan encoding

```
kategori = df_copy.select_dtypes(include='object').columns
print(f'Kolom kategori: {list(kategori)}')
```

Kolom kategori: ['Brand', 'model', 'Transmission', 'Owner', 'FuelType']

- Mengecek isi dari setiap kolom kategorikal

```

▶ for col in kategori:
    unique_counts = df_copy[col].nunique()
    print(f'Kolom {col} memiliki {unique_counts} kategori')

    if unique_counts < 10:
        print(f'Isinya {df_copy[col].unique()}')
    else:
        print(f'Isinya terlalu banyak')
    print('-' * 30)

```

```

... Kolom Brand memiliki 33 kategori
    Isinya terlalu banyak
    -----
    Kolom model memiliki 306 kategori
    Isinya terlalu banyak
    -----
    Kolom Transmission memiliki 2 kategori
    Isinya ['Manual' 'Automatic']
    -----
    Kolom Owner memiliki 2 kategori
    Isinya ['first' 'second']
    -----
    Kolom FuelType memiliki 3 kategori
    Isinya ['Diesel' 'Petrol' 'Hybrid/CNG']
    -----

```

- Untuk kolom kategorikal yang datanya ordinal (memiliki tingkatan dari setiap value), kita menggunakan map encoding. Di dataset kita yang memiliki data ordinal adalah kolom Owner. Value 'first' biasanya lebih baik daripada value 'second'

```

# Map encoding ----> Untuk data yang ordinal. Kenapa tidak menggunakan Label Encoding? karena kalau label encoding itu akan memberikan angka sesuai abjad.
owner_map = {
    'first': 1,
    'second': 2
}

df_copy['Owner_Encode'] = df_copy['Owner'].map(owner_map)

```

Setelah di encode, kita hapus kolom 'Owner'

```

▶ df_copy = df_copy.drop(columns='Owner')

```

- Untuk fitur kategorikal yang memiliki kategori yang banyak, kita menggunakan frequency encoding. Dataset kita yang memiliki kategori yang banyak adalah kolom 'Brand' dan 'Model'

```
# Frequency encoding ----> Untuk data yang memiliki kategori yang banyak
freq_brand = df_copy['Brand'].value_counts().to_dict()
df_copy['Brand_Freq'] = df_copy['Brand'].map(freq_brand)

freq_model = df_copy['model'].value_counts().to_dict()
df_copy['Model_Freq'] = df_copy['model'].map(freq_model)

df_copy = df_copy.drop(columns=['Brand', 'model'])

df_copy.head()
```

	Age	kmDriven	Transmission	FuelType	AskPrice	Owner_Encode	Brand_Freq	Model_Freq
2	14	77246.0	Manual	Diesel	184999	1	237	70
3	7	83500.0	Manual	Diesel	565000	2	2293	238
4	5	45000.0	Automatic	Petrol	685000	1	2293	151
5	10	83000.0	Automatic	Diesel	1350000	1	114	10
6	10	168000.0	Manual	Diesel	1025000	2	410	60

- Untuk fitur kategori yang datanya nominal, kita menggunakan one hot encoding.

```
df_copy = pd.get_dummies(df_copy, columns=['Transmission', 'FuelType'], drop_first=True, dtype=int)

df_copy.head()
```

	Age	kmDriven	AskPrice	Owner_Encode	Brand_Freq	Model_Freq	Transmission_Manual	FuelType_Hybrid/CNG	FuelType_Petrol
2	14	77246.0	184999	1	237	70	1	0	0
3	7	83500.0	565000	2	2293	238	1	0	0
4	5	45000.0	685000	1	2293	151	0	0	1
5	10	83000.0	1350000	1	114	10	0	0	0
6	10	168000.0	1025000	2	410	60	1	0	0

Next steps: [Generate code with df_copy](#) [New interactive sheet](#)

- Membagi data latih dengan data uji menggunakan library train_test_split

```
from sklearn.base import TransformerMixin
X = df_copy.drop(columns='AskPrice')
y = df_copy['AskPrice']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print(f'Total data asli: {df_copy.shape}')
print(f'Total data latih: {X_train.shape}')
print(f'Total data uji: {X_test.shape}')
```

```
... Total data asli: (7056, 9)
Total data latih: (5644, 8)
Total data uji: (1412, 8)
```

Pada kasus ini, saya mengatur untuk data uji itu 0.2 atau 20% dari data asli dan untuk data latih 80% dari data asli.

- Melakukan scaling menggunakan standard scaler. Metode ini digunakan agar data kita memiliki nilai rata-rata = 0 dan nilai standar deviasi = 1

```
▶ scaler = StandardScaler()  
  X_train_scaled = scaler.fit_transform(X_train)  
  X_test_scaled = scaler.transform(X_test)
```

- Membuat model random forest regressor

```
▶ rf_model = RandomForestRegressor()  
  rf_model.fit(X_train_scaled, y_train)
```

... ▾ RandomForestRegressor ⓘ ?
RandomForestRegressor()

- Melakukan prediksi menggunakan model yang telah kita buat. Kemudian melakukan evaluasi menggunakan mean absolute error dan r2 score

```
▶ y_predict_rf = rf_model.predict(X_test_scaled)  
  
mae_rf = mean_absolute_error(y_test, y_predict_rf)  
r2_rf = r2_score(y_test, y_predict_rf)  
  
print(f'Hasil evaluasi Random Forest Regressor')  
print('=' * 30)  
print(f'Mean Absolute Error = {mae_rf:,.0f}')  
print(f'R2 Score = {r2_rf:.2f}')
```

... Hasil evaluasi Random Forest Regressor
=====

Mean Absolute Error	= 121,259
R2 Score	= 0.84

Maksud dari hasil evaluasi diatas adalah model berhasil menebak pola data sekitar 0.84 atau 84%. Untuk model kita rata-rata selisih errornya dengan dengan data asli adalah 121.259 rupe (dataset diambil dari orang India). Angka ini bisa kemurahan atau mungkin kemahalan.

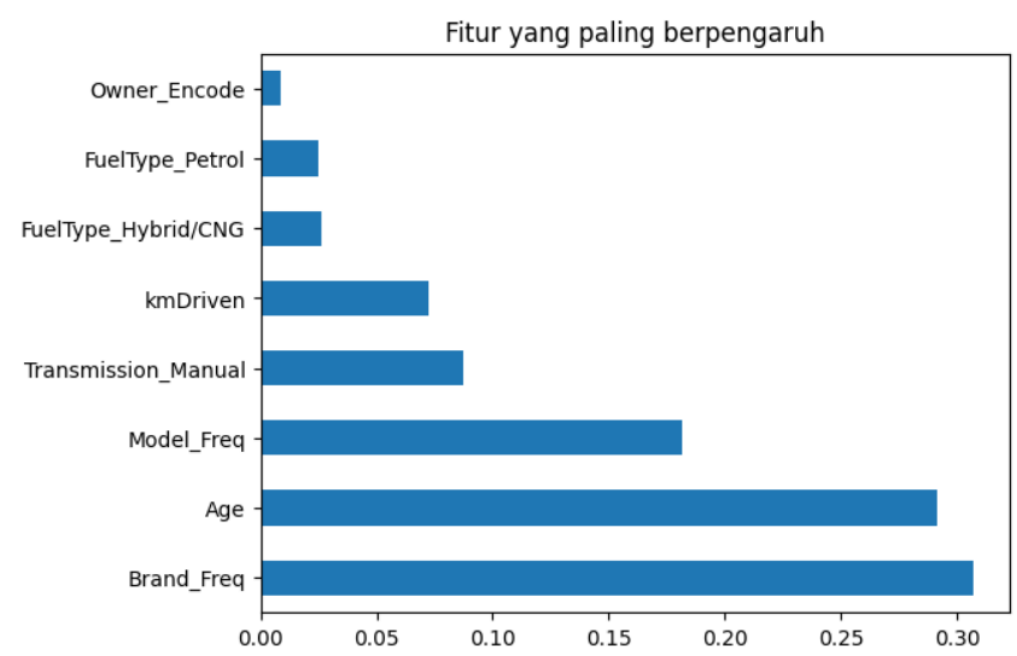
- Mencari fitur yang paling berpengaruh dari dataset yang kita miliki

```
fi = pd.Series(rf_model.feature_importances_, index=X.columns)

print(fi.nlargest(10))

fi.nlargest(10).plot(kind='barh')
plt.title('Fitur yang paling berpengaruh')
plt.show()
```

...	Brand_Freq	0.306952
	Age	0.291137
	Model_Freq	0.181791
	Transmission_Manual	0.087522
	kmDriven	0.072459
	FuelType_Hybrid/CNG	0.026445
	FuelType_Petrol	0.025152
	Owner_Encode	0.008542
	dtype: float64	



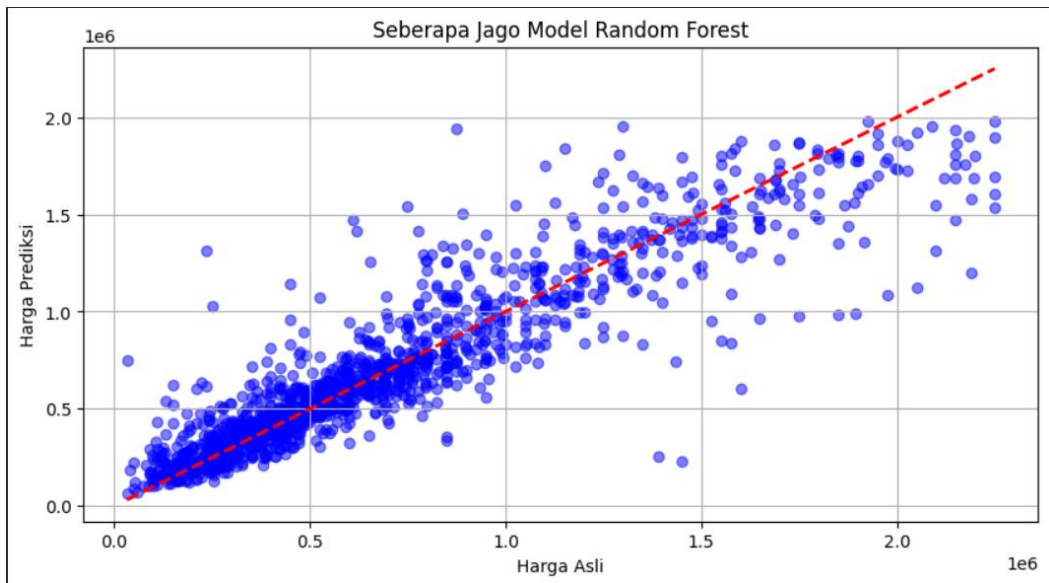
Ternyata, fitur yang paling berpengaruh adalah Brand_Freq. Maksudnya, semakin terkenal brand yang dimiliki si mobil, kemungkinan harganya akan semakin tinggi.

- Membuat grafik scatter untuk melihat seberapa bagus model kita

```
plt.figure(figsize=(10, 5))
plt.scatter(y_test, y_predict_rf, alpha=0.5, color='blue')

min_val = min(y_test.min(), y_predict_rf.min())
max_val = max(y_test.max(), y_predict_rf.max())

plt.plot([min_val, max_val], [min_val, max_val], color='red', linestyle='--', linewidth=2)
plt.xlabel('Harga Asli')
plt.ylabel('Harga Prediksi')
plt.title('Seberapa Jago Model Random Forest')
plt.grid(True)
plt.show()
```



Ternyata, model kita jago menebak mobil yang harganya murah, untuk mobil yang harganya mahal, model seringkali salah tebak

- Melihat korelasi antara harga dengan umur mobil

```
korelasi = df[['Age', 'AskPrice']].corr().iloc[0, 1]
print(f'Korelasi Umur dengan Harga Mobil : {korelasi:.2f}')
```

Korelasi Umur dengan Harga Mobil : -0.30

Ternyata korelasinya -0.30 yang artinya semakin tua mobil, maka semakin murah harganya. Namun, korelasi ini tidak terlalu kuat karena tidak mendekati -1 atau 1.

```
plt.figure(figsize=(10, 5))
sns.lineplot(x='Age', y='AskPrice', data=df_copy, marker='o', color='red')
plt.title('Grafik Age vs Harga')
plt.xlabel('Umur Mobil')
plt.ylabel('Harga Mobil')
plt.grid(True)
plt.show()
```

